How the SFU Guide Car finds its way

A plain-English walk-through of the navigation software: what each part does, which tools it uses, and how far we have got. Technical names are in brackets so you can look them up.

The big picture

The car guides a student to a room, at walking pace. Its software is a chain of steps. Each step hands its result to the next, and every driving instruction passes one last safety check before it reaches the wheels.

Step	What it does	Main tools
1. Hear	Understands which room the student wants and confirms it out loud	ElevenLabs speech-to-text (STT) and voice (TTS); Gemini for conversation
2. Route	Finds the best way there on SFU's own floor plans	SFU campus map data (GIS); shortest-path search
3. Manage the trip	Breaks the route into pieces: drive, door, elevator; speaks directions	Our mission program (ROS 2 node)
4. Know where it is	Works out the car's position from cameras and the phone	Visual odometry and mapping (RTAB-Map); sensor fusion (EKF)
5. See obstacles	Spots walls, floor edges, people and objects	Stereo depth; floor detection (SegFormer); object detection (YOLO)
6. Plan and steer	Plans a path the car can actually drive and follows it	Navigation stack (Nav2): Hybrid-A* planner, Pure Pursuit controller
7. Safety check	Caps speed, stops for people, reacts to bumps	Our velocity gate
8. Move	Turns the instruction into motor signals	Laptop to ESP32 microcontroller over USB

Everything above runs on a laptop riding on the car, using the robot software framework **ROS 2** (Robot Operating System). Each step is a separate small program (a node); they pass messages to each other (topics).

1. Hearing the student

- The laptop microphone records the student. Recording starts by itself when the voice rises above the room's background noise and stops after a short pause (voice-activity detection), so nobody presses a key.
- Quiet clips are turned up before sending, and the sound goes to **ElevenLabs**, which turns speech into text (speech-to-text, STT; its model is called Scribe). We give it campus words such as “AQ” and “ASB” as hints.
- Our room finder reads the room out of the sentence (for example “take me to A Q thirty-one twenty-eight” becomes AQ 3128) and checks it really exists on the map. If it doesn't, the car suggests the nearest real rooms. It also understands kinds of place, like “a washroom”.
- The car reads the room back in a natural voice (ElevenLabs text-to-speech, TTS) and waits for “yes”. If the internet drops, a voice that runs on the laptop takes over (Piper).

- For open conversation (questions, changing your mind) the car's voice agent uses Google's **Gemini** language model. The demo's confirm-and-go mode does not need it.

2. The campus map and the route

- We downloaded SFU's own floor plans from the university's online campus map (a GIS, geographic information system, run on ArcGIS): every room, corridor, door, elevator and staircase, floor by floor, for the Academic Quadrangle (AQ), Applied Sciences Building (ASB), Shrum Science Centre (SCK) and others.
- From these we built a **route graph**: points along corridors, at doorways and at elevators, joined by lines with their real lengths.
- To get somewhere, the program searches that graph for the shortest way (Dijkstra's algorithm). It adds a 15-metre “cost” for every door, because each door means stopping to ask for help, and 60 m for cutting through an office or lab, so it prefers corridors.
- The route is then split into **legs**: drive along one floor, go through a door, take an elevator.

3. Managing the trip

- Our mission program is in charge from “yes” to arrival. For a drive leg it hands the list of waypoints to the navigation system (step 6).
- At a door or elevator it stops and asks: “Could you hold the door, please?” or “Please press the button for level 3.” After an elevator it switches to that floor's map.
- It speaks directions on the way, and the student can say pause, carry on or cancel.
- It checks the student is still following: the phone web app sends a heartbeat every second over campus Wi-Fi, and Bluetooth signal strength (RSSI) is a second check. The rear camera can also track the student (person tracking with ByteTrack). If the student falls behind, the car stops and asks “Are you still with me?”

4. Knowing where the car is (localization)

This is the hardest part, because the car has no wheel sensors (encoders) to count how far the wheels have turned. Instead it works out its movement from what it sees and feels:

- **Two front cameras, like eyes** (a stereo pair, 12.6 cm apart). By tracking the same points from one picture to the next, the software measures how the car moved (visual odometry). Because two cameras see depth, these measurements are in real metres.
- **The phone on the car**: its gyroscope gives how fast the car turns, and its accelerometer feels bumps. An app (Sensor Logger) streams this to the laptop over Wi-Fi.
- **Combining them**: a standard sensor-fusion filter (extended Kalman filter, EKF, from the robot_localization package) blends camera and phone readings 30 times a second into one smooth estimate.
- **Recognising places**: each floor is mapped in advance by walking it with the cameras (RTAB-Map, “real-time appearance-based mapping”). While guiding, RTAB-Map recognises views it has seen before and corrects the drift that builds up (loop closure).
- **Reading room numbers**: when the camera reads a number on a door sign (text recognition, OCR, with PaddleOCR), the car knows it is beside that door. It only trusts a reading if that door is within 8 m of where it thinks it is.

- **Corridor direction**: long straight corridors have lines that meet at one point in the picture (the vanishing point); that tells the car which way it is facing.
- **Outdoors**: the phone's GPS and compass take over. A switch-over program changes sources at building entrances and loads the right floor map.
- **Lining the maps up**: our tool matches each camera-made map to SFU's floor plan using three or more doors, so “AQ 3128” means the same spot in both.

5. Seeing obstacles

- **Depth from the two cameras** (stereo matching): how far away everything in front is.
- **Where the floor is**: an AI model labels each pixel as floor or not (semantic segmentation with SegFormer). Where the floor ends in front of the car, there is a wall or obstacle.
- **What things are**: an AI object detector (YOLO11, “You Only Look Once”) draws boxes around people, chairs, benches, bags and other objects. We also built a version that knows school objects such as doors, elevators, desks and whiteboards (YOLOE with chosen class names).
- All of this goes onto a grid map of “no-go” cells around the car (a costmap), sized to the car's real body, 75 × 41 cm (its footprint).

6. Planning a path and steering along it

- We use the standard robot navigation system for ROS 2 (**Nav2**).
- The car steers like a normal car (front-wheel steering, Ackermann steering), so it cannot turn on the spot. The path planner therefore only draws paths a car can drive: smooth curves no tighter than a 1-metre turning circle (Smac Hybrid-A* planner).
- A path-following program (Regulated Pure Pursuit controller) aims at a point a little way along the path, ten times a second, and slows down in tight bends and near obstacles.

7. The safety check (velocity gate)

Every driving instruction passes this filter last, whichever program produced it:

- Speed limit: walking pace, at most 0.6 m/s indoors (1.0 m/s outdoors).
- People ahead: starts slowing at 3 m and stops at 1 m.
- Bumps: a sudden jolt on the phone's accelerometer (over 5 m/s^2), or the car being told to move while the cameras say it is not moving (stalled). Either blocks forward driving for 3 seconds (reversing is still allowed) and the car says “Sorry, I bumped into something.”
- Stops if the phone link is lost or driving has not been switched on; speeds change gradually, never with a jerk.

8. Reaching the motors

- The laptop converts “speed and turn” into steering and throttle numbers between −1 and 1, and sends them to a small microcontroller board (**ESP32**) over USB, 20 times a second, as short text lines (“C <steering> <throttle>”).
- On the full car, the ESP32 imitates the car's hand-held radio controller: its analog outputs (DAC, digital-to-analog converter) set the controller's steering and throttle voltages, and the controller radios the car.

- For the demo, the ESP32 drives the motors directly through a motor-driver board (L298N), using speed signals (PWM, pulse-width modulation) on pins 25 and 13.
- Built-in safety (watchdog): if no instruction arrives for 0.3 seconds — laptop crash, unplugged cable — the ESP32 stops the motors by itself.

How we tested it

- **A full simulator** (hardware-in-the-loop, HIL): the real ESP32 program runs on the laptop against a simulated car, simulated cameras and a simulated corridor loop. The whole chain, from destination to wheels, drove complete simulated trips, including recognising places on the second lap.
- **Camera calibration**: 56 good photo pairs of a printed 25 mm checkerboard (stereo calibration with OpenCV). Measurement error about half a pixel; the lens spacing came out at 12.7 cm against 12.6 cm measured with a ruler, and we fixed it to the ruler's value.
- **Real walks**: we pushed the camera rig along three routes: twice round the AQ 3000 level (once each way) and once from the AQ through SCK to the ASB 9000 level, with the phone recording at the same time, and lined up the video and phone clocks to a few milliseconds (cross-correlation of the turning rate seen by each).
- **Object detection** was run over all three walks; **voice** was tested on the laptop with confirmation by voice; the **motor program** was tested in the simulator (forward, reverse, full speed, watchdog stop).
- **Automated tests**: the voice, safety-gate and perception code each have their own test suites that run on every change.

Where we are now

Part	Status
Voice: hear the room, check it, confirm by voice	Working on the laptop
Voice “yes” starts the motors (demo)	Working in simulation; being tried on the car
Camera calibration	Done
Object detection and the judges' live screen	Working on recorded walks; live run to be checked
Whole navigation chain (map → route → path → safety → motors)	Built; tested in the simulator
Real floor maps from our walks (RTAB-Map)	Walks recorded; maps not built yet (videos uploading)
Steering on the real car	Broken (hardware); the demo drives straight only
Full self-guided trip on campus	Not yet: needs real maps and working steering

Glossary

Term	Meaning
ROS 2	Robot Operating System: the framework our programs run in and talk through
Node / topic	One small program / a named channel programs send messages on
GIS / ArcGIS	Geographic information system: SFU's online campus map we took the floor plans from
STT / TTS	Speech-to-text / text-to-speech
Visual odometry	Measuring how the car moved from successive camera pictures
Stereo	Two cameras side by side, giving depth like two eyes
EKF	Extended Kalman filter: blends several noisy sensors into one best estimate
RTAB-Map	Real-Time Appearance-Based Mapping: builds maps from cameras and recognises places
Loop closure	Recognising a place seen before and correcting the built-up error
OCR	Optical character recognition: reading text in images
YOLO	“You Only Look Once”: a fast AI object detector
Segmentation	Labelling every pixel (here: floor or not floor)
Costmap / footprint	Grid of no-go cells around the car / the car's outline used in it
Nav2	The standard ROS 2 navigation system (planning and path following)
Ackermann steering	Car-style front-wheel steering: cannot turn on the spot
Hybrid-A*	A path search that only produces paths a car can actually drive
Pure Pursuit	Steering by aiming at a point a short way along the path
ESP32	A small, cheap microcontroller board with USB, Wi-Fi and analog outputs
DAC / PWM	Digital-to-analog converter (sets a voltage) / pulse-width modulation (sets motor speed)
L298N	A common two-motor driver board
Watchdog	A timer that stops the motors if instructions stop arriving
HIL simulation	Hardware-in-the-loop: real control program, simulated car and world